SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
IILM UNIVERSITY, GREATER NOIDA

SYSTEM ARCHITECTURE DOCUMENT

Digital Twin of a University Building for
Occupancy-Aware Energy and Indoor Environmental Monitoring

CampusTwin EnergyMind

Item	Value
Document ID	CTEM-SAD-001
Version	1.0
Date	20 September 2026
Project Type	B.Tech Major Project
University	IILM University, Greater Noida
Academic Session	2026–27
Architecture Status	Academic architecture baseline
Primary Implementation Baseline	SRS v1.0
Guide	Mrs. Bhavana Srivastava, Assistant Professor

Source basis: PRD, SRS and Phase-I Project Report. Confirmed, Proposed and TBD architecture decisions are explicitly distinguished.

Document Control & Contents

Attribute	Value
Primary source	SRS — implementation requirements baseline
Product baseline	PRD — product/features, targets and explicit non-goals
Concept/technology baseline	Phase-I Report — 3D concept, IoT and proposed technology baseline
Current MVP	SVH, Engineering Block, Law Block; extensible building model
Control boundary	Monitoring and decision support; no autonomous physical control
Unresolved architecture decision	2D MVP vs 3D Digital Twin visualization — TBD

1. Introduction
2. Architecture Overview
3. Architecture Style and Design Approach
4. Technology Stack
5. System Components
6. Frontend Architecture
7. Backend Architecture
8. Database Architecture
9. Building and Digital Twin Data Model
10. API Architecture
11. Data Flow
12. Occupancy, Energy and Environmental Data Processing
13. Energy-Waste Detection Architecture
14. Authentication and Authorization Architecture
15. Security Architecture
16. Data Storage and Retention
17. Deployment Architecture
18. Monitoring and Logging
19. Performance and Scalability
20. Error Handling and Resilience
21. Architecture Decisions and Trade-Offs
22. Scalability and Future Extensibility
23. Final Architecture Summary

1. INTRODUCTION

1.1 Purpose

This System Architecture Document (SAD) describes how CampusTwin EnergyMind is organized technically to satisfy the requirements defined in the SRS and the product capabilities defined in the PRD. It provides the component structure, technology boundaries, communication paths, data model, security approach, deployment model, and scalability approach needed for implementation.

1.2 Project Overview

CampusTwin EnergyMind is a web-based monitoring and decision-support prototype that associates room-level occupancy, indoor environmental conditions, energy consumption, HVAC state, and lighting state with a digital representation of university building spaces. It supports current-state visibility, historical analytics, energy-waste detection, alerts, and recommendations. The initial implementation may use simulated telemetry and can later accept selected real sensor data.

1.3 Architecture Objectives

- Keep frontend, backend, processing and persistence concerns clearly separated.
- Support simulated telemetry and future ESP32/MQTT integration through a common data model.
- Support the current three-building MVP without hard-coding the building structure into the application.
- Provide secure authenticated access and role-based protection for administrative capabilities.
- Support the documented 2-second dashboard update target, 30-day chart target, 500-room/5-second ingestion target and 99% operational-hours availability target.
- Keep the system modular enough for additional buildings, rooms, sensors and analysis capabilities without fundamental architectural redesign.
- Keep implementation practical for a B.Tech project and avoid unnecessary enterprise infrastructure.

1.4 Architecture Scope

The architecture covers the React web application, Digital Twin/spatial presentation layer, FastAPI backend, validation and business processing, PostgreSQL persistence, simulation, REST-based application ingestion, optional MQTT/ESP32 integration, analytics, waste detection, alerts, recommendations, authentication/RBAC, security, deployment, monitoring and future extensibility.

1.5 Reference Documents

Reference	Role in Architecture
Product Requirements Document (PRD)	Product/feature baseline, MVP goals, user stories, performance targets, data entities, workflow and explicit non-goals.
Software Requirements Specification (SRS) v1.0	Primary implementation baseline: three-building MVP, functional/non-functional requirements, validation, security, acceptance and test coverage.
Major Project Report (Phase-I)	Project concept, research context, 3D Digital Twin direction, IoT/ESP32/MQTT approach and technology feasibility.

2. ARCHITECTURE OVERVIEW

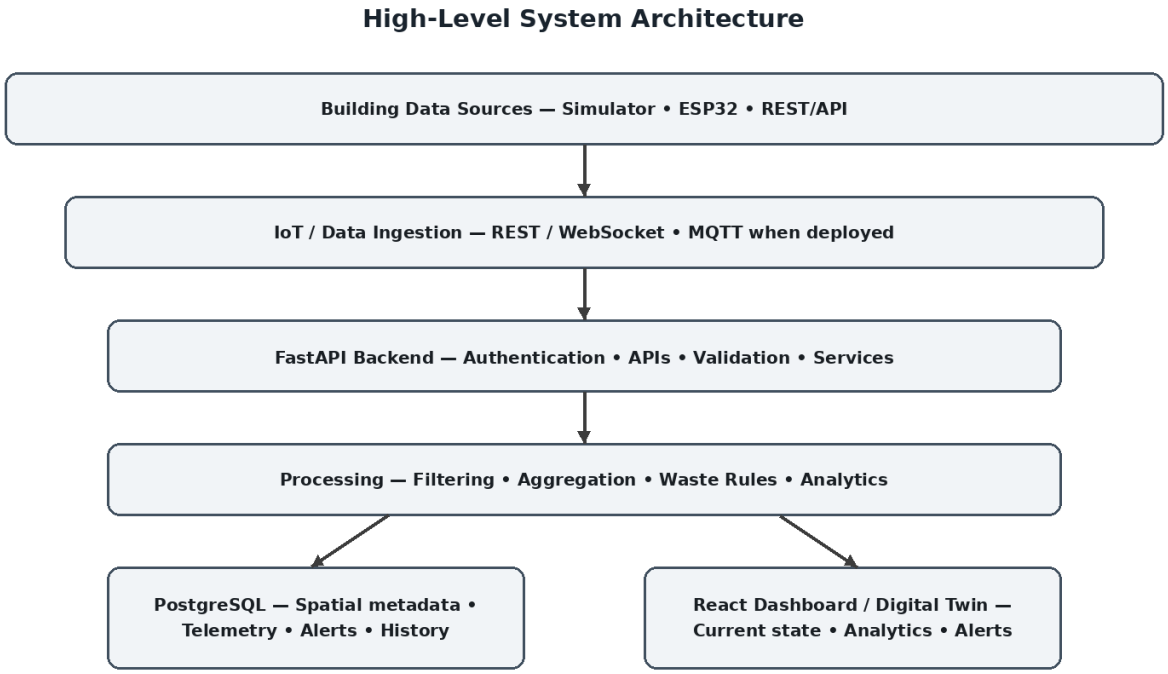


Figure 1. High-level system architecture.

The architecture uses a simple request-and-data-processing model. Data enters through the simulator or supported IoT/API interfaces, is validated and processed by FastAPI services, persists in PostgreSQL, and is then exposed to the React dashboard and Digital Twin presentation. Analytics and waste rules operate inside the backend/application layer rather than as separate distributed services.

Layer	Responsibility
Data Source / Ingestion	Generate or receive room telemetry from simulator, REST/API sources, or ESP32/MQTT when deployed.
Backend API	Authenticate users, authorize operations, validate inputs, expose application services and retrieve/store data.
Processing	Apply filtering, aggregation, correlation, waste detection and recommendation rules.
Persistence	Store spatial metadata, user records, telemetry, resource states, alerts, recommendations and history.
Presentation	Render dashboard, building/floor/room state, charts, alerts and recommendations according to role.

2.1 Architectural Boundary

The system ends at decision-support outputs. It does not issue actuator commands to HVAC units, lighting circuits, breakers or other critical physical infrastructure. A future BMS integration would require a separately approved control architecture.

3. ARCHITECTURE STYLE AND DESIGN APPROACH

The recommended style is a layered, modular web application with a modular monolithic FastAPI backend. The backend may contain clear internal modules for authentication, ingestion, validation, analytics, waste detection

and recommendations, but these modules do not need to become independent microservices for the current MVP.

Principle	Application
Layered separation	React presentation, FastAPI API, processing/services, data access and PostgreSQL are separated.
Modularity	Building, room, telemetry and rule processing are reusable modules.
API-driven communication	Frontend communicates with backend through authenticated APIs; telemetry can use REST and MQTT adapters.
Configuration over hard-coding	Buildings, floors and rooms are stored as records, allowing additional spaces without changing core logic.
Hybrid data strategy	Simulator and real sensor data use compatible telemetry structures.
Fail-safe decision support	Invalid/missing data produces safe unavailable states; recommendations never become automatic control commands.
Practical deployment	A small set of deployable components is preferred over microservices, Kubernetes or an event-bus platform.

3.1 Why a Modular Monolith

The project has a bounded scope, a single application domain and a student development team. A modular monolith reduces deployment and debugging complexity while preserving clean internal separation. Microservices would add network boundaries, deployment overhead, service discovery and observability requirements that are not necessary for the current MVP.

3.2 Patterns Not Selected for MVP

Pattern	Decision	Reason
Microservices	Not selected	Unnecessary operational complexity for the current scope.
Kubernetes	Not selected	No cluster-orchestration requirement.
Enterprise event bus	Not selected	MQTT plus application APIs are sufficient for the current telemetry model.
Heavy BIM/IFC pipeline	Not required for MVP	PRD supports simplified SVG/JSON layouts; 2D/3D representation remains a scope decision.

4. TECHNOLOGY STACK

Layer	Technology	Purpose	Status
Frontend	React.js	Web dashboard, navigation, visualization and analytics UI.	Confirmed
Digital Twin	Three.js / React Three Fiber	3D visualization if the confirmed direction requires 3D.	Phase-I supported; TBD for final MVP
Spatial layout	SVG / JSON	Lightweight floor/room representation.	PRD/SRS supported
Backend	Python + FastAPI	APIs, authentication, validation, business logic and processing.	Confirmed
Database	PostgreSQL	Structured metadata, telemetry, alerts, recommendations and history.	Confirmed
IoT device	ESP32	Selected real sensor acquisition	Phase-I / Optional

Layer	Technology	Purpose	Status
		where available.	
IoT transport	MQTT	Telemetry transport for ESP32 integration.	Phase-I / Optional
Analysis	Python + Scikit-learn	Analysis and future occupancy prediction.	Phase-I / Future/Proposed
Simulation	Python-based simulator	Synthetic occupancy and telemetry generation.	Confirmed capability; implementation TBD
Web foundation	HTML/CSS/JavaScript	Standard web application implementation.	Confirmed

No unrelated technology is required by the source baseline. Supporting libraries may be selected during implementation only where needed and should be recorded as Proposed.

5. SYSTEM COMPONENTS

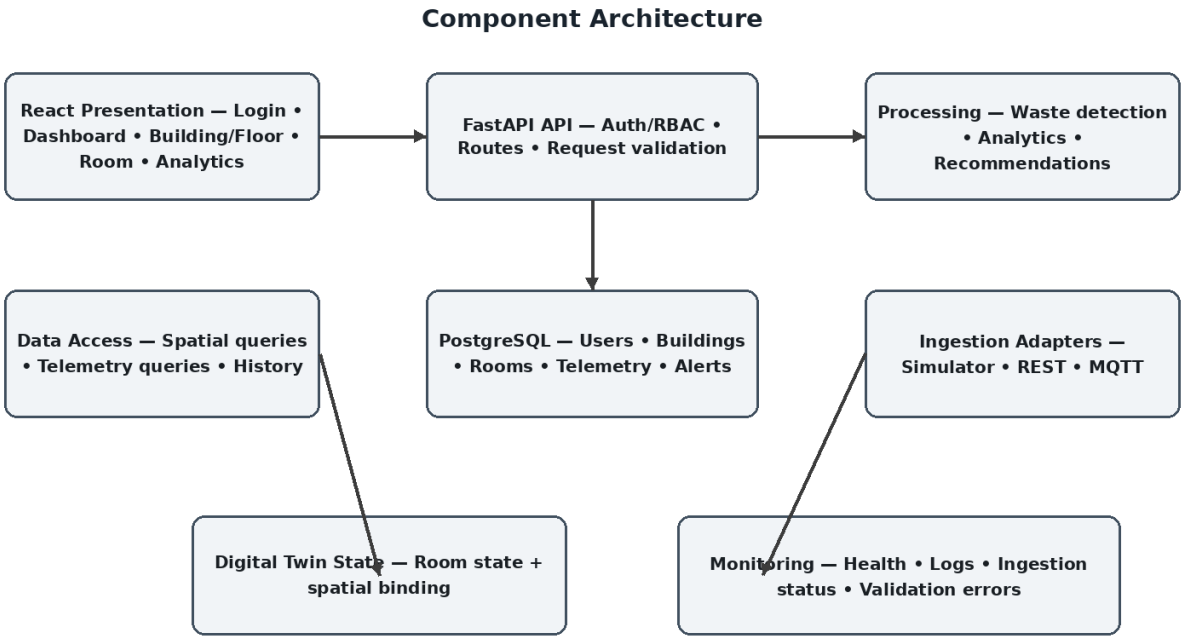


Figure 2. Component architecture.

5.1 React Frontend

Provides login, dashboard, building/floor navigation, room detail views, occupancy/environment/energy displays, alerts, recommendations and historical analytics. It requests data from FastAPI rather than accessing PostgreSQL directly.

5.2 Digital Twin Visualization

Binds Building → Floor → Room records to a spatial representation and applies current status overlays. The PRD specifies a 2D MVP; the Phase-I report describes a 3D Digital Twin using Three.js/React Three Fiber. The final renderer is therefore TBD pending project-team/guide confirmation.

5.3 FastAPI Backend

Acts as the application boundary. It exposes authenticated APIs, validates input, applies business rules, manages current state, retrieves historical data, generates alerts and recommendations, and coordinates ingestion.

5.4 PostgreSQL

Stores spatial metadata and time-stamped telemetry. It is the persistent store for users, buildings, floors, rooms, occupancy, environmental data, energy, resource state, alerts and recommendations.

5.5 Data Simulation Engine

Produces controlled synthetic occupancy, temperature, humidity, CO_2, energy, HVAC and lighting values following realistic university usage patterns. It should use the same logical ingestion schema as real telemetry.

5.6 IoT Integration

ESP32 devices can publish selected measurements through MQTT. The adapter maps incoming messages to configured room/stream identifiers before validation and storage. Real deployment depends on hardware, network and permissions.

5.7 Analytics and Waste Detection

Backend services correlate occupancy with resource state and energy. Configured rules identify potential waste, create alerts and produce human-readable recommendations; historical analytics supports occupancy/energy comparisons.

6. FRONTEND ARCHITECTURE

The frontend is organized as a feature-oriented React application. The exact file structure is implementation-level and is therefore recommended rather than finalized.

React Area	Responsibilities
Authentication	Login, session state and protected UI behavior.
Dashboard	Summary cards, selected building context, active alerts, system health and navigation.
Building View	Spatial map/twin rendering, building selection and room status visualization.
Floor Navigation	Select configured floors and load their room layout.
Room Details	Room ID, capacity if known, occupancy, environment, energy, HVAC, lighting and recent trends.
Occupancy View	Current state, headcount where available and historical occupancy.
Environmental Monitoring	Temperature, humidity and CO_2 current/historical values.
Energy Monitoring	kW/kWh values, standby comparison where configured and trends.
Alerts	Filterable active/resolved waste alerts and evidence.
Recommendations	Decision-support issue, metric, action, severity, timestamp and status.
Historical Analytics	24-hour, 7-day and 30-day charts and utilization views.

6.1 Component Hierarchy

React Application → Authentication → Dashboard → Building View → Floor Navigation → Room Details → Occupancy / Environment / Energy → Alerts / Recommendations → Historical Analytics.

6.2 State and API Communication

For the current MVP, local React state and small reusable hooks/services are sufficient. A global state-management library is not required by the source documents and is therefore not introduced. API communication should use a centralized request layer so authentication headers, error handling and response parsing remain consistent.

6.3 UI State Handling

State	Architecture Behavior
Loading	Show a local skeleton/spinner without blocking unrelated dashboard sections.
Empty	Explain that no data is available for the selected room/time range.
Stale/missing telemetry	Show the last valid timestamp where available and mark the metric unavailable/stale.
API error	Show a controlled error and retry option; never expose internal stack traces.
Forbidden	Hide/disable unavailable actions and enforce the same permission at the backend.

7. BACKEND ARCHITECTURE

Module	Responsibilities
Authentication	Credential verification, authentication context and logout.
API Routes	HTTP request/response schemas and orchestration.
Validation	Required fields, room association, timestamps, numeric values, states and duplicate detection.
Services / Business Logic	Spatial hierarchy, current state, analytics, rule evaluation and workflows.
Waste Detection	Occupancy/resource/energy correlation and configured duration/threshold rules.
Recommendations	Generate evidence-linked decision-support records.
Analytics	Historical aggregation, occupancy-energy comparison and utilization.
Ingestion	Simulator and REST ingestion; MQTT adapter when enabled.
Data Access	Parameterized queries/repository functions and transaction handling.
Error Handling	Consistent validation, authorization, dependency and service error responses.

7.1 Request Processing Path

Client Request → Authentication/Authorization → Input Validation → Business Service → Data Access / Processing → Database or Decision Output → Structured Response → React UI.

7.2 Recommended Backend Structure

fastapi_app/ → auth/ → api/ → schemas/ → services/ → ingestion/ → analytics/ → waste_detection/ → recommendations/ → data_access/ → database/ → configuration/. This is a recommended organization, not a finalized source-defined folder structure.

8. DATABASE ARCHITECTURE

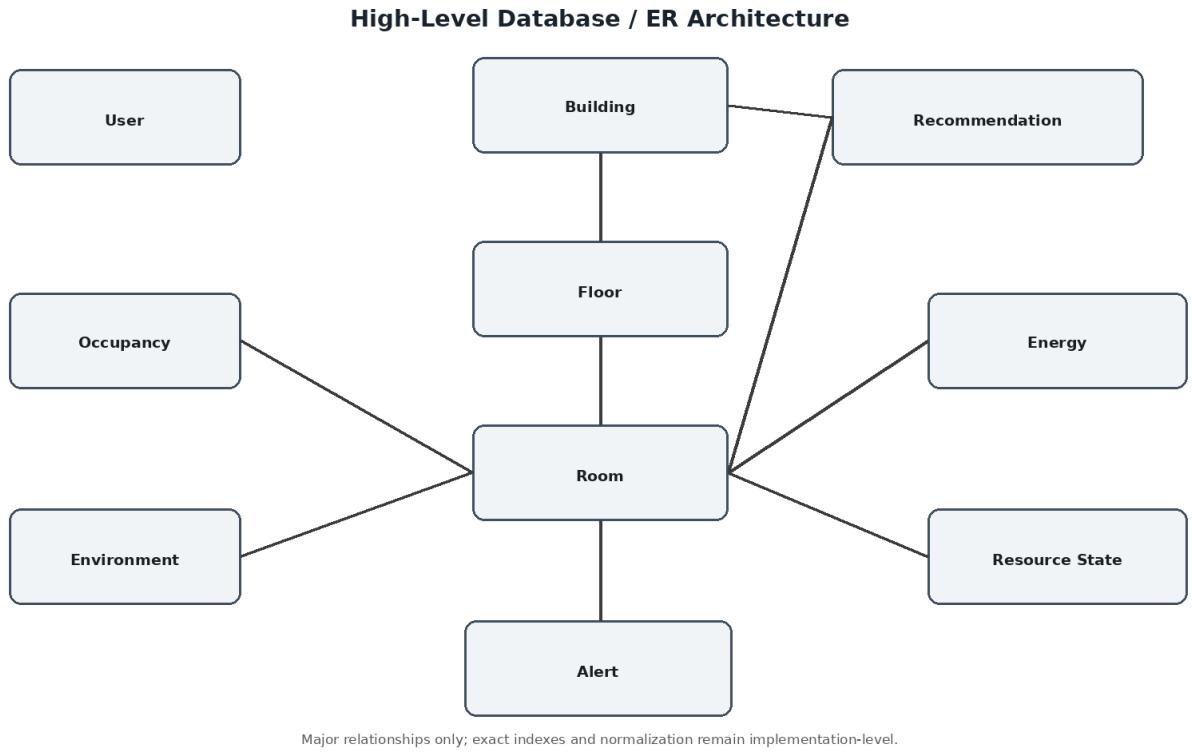


Figure 3. High-level database / ER architecture.

Entity	Architectural Role	Key Relationships
User	Authenticated account and role context.	Role controls protected application/API operations.
Building	Top-level spatial unit.	One building has many floors.
Floor	Building subdivision and spatial map.	Many floors belong to one building; one floor has many rooms.
Room	Primary monitoring unit.	Room belongs to one floor; telemetry/resource states reference room.
Occupancy	Time-stamped aggregate presence/count.	Many records belong to one room.
Environmental Data	Temperature, humidity and CO_2 measurements.	Many records belong to one room.
Energy Data	Time-stamped kW/kWh values.	Many records belong to one room.
Resource State	Time-stamped HVAC/lighting states.	Many records belong to one room.
Alert	Detected event requiring attention.	References affected room and evidence/time.
Recommendation	Decision-support output.	References affected room and supporting evidence.

8.1 Data Access Principles

- Use primary keys for major entities and foreign keys for spatial/telemetry relationships.
- Use parameterized database operations to reduce injection risk.
- Index frequent room/timestamp telemetry queries.
- Use aggregated historical queries instead of transferring unnecessary raw telemetry to the browser.
- Keep current state logically distinct from historical records where this improves dashboard performance.

9. BUILDING AND DIGITAL TWIN DATA MODEL

The architecture treats spatial configuration as data: University → Building → Floor → Room → Telemetry / Resource State.

Building	Floor	Room IDs
SVH — Sir Vishveshwarya Hall	Ground	101–112
SVH — Sir Vishveshwarya Hall	Second	201–213
Engineering Block	Ground	101–108
Engineering Block	First	201–208
Engineering Block	Second	301–308
Engineering Block	Third	401–408
Law Block	Ground	101–108
Law Block	First	201–208
Law Block	Second	301–308
Law Block	Third	401–408

The supplied room IDs are preserved exactly. No additional rooms are introduced. Where physical floor naming and room numbering require clarification, the mapping remains a configuration/TBD item rather than an inferred architectural fact.

9.1 Extensibility Model

A future building is added by inserting a Building record, its Floor records, its Room records and corresponding telemetry streams. The React UI reads these records rather than containing building-specific navigation logic.

9.2 2D MVP vs 3D Digital Twin — Architecture Decision

Source / Status	Architecture Treatment
PRD — Confirmed product baseline	Interactive 2D spatial floor layout is the MVP representation; complex 3D/WebGL/VR is explicitly outside the PRD MVP.
Phase-I Report — Proposed concept/technology baseline	3D Digital Twin with React.js + Three.js/React Three Fiber is described.
SRS — Explicit clarification	The discrepancy is retained and final visual implementation direction is TBD.
Architecture response	The spatial data model, telemetry binding and backend APIs are representation-neutral; a 2D or 3D renderer can consume the same room state after confirmation.

10. API ARCHITECTURE

Exact API endpoint names are not finalized in the source documents. The following is therefore a Recommended API Design — Proposed.

Category	Purpose	Input	Output	Auth.	Validation
Authentication	Login/logout/session.	Credentials/session.	Auth result/status.	Protected as applicable.	Credential validation.
Building	Retrieve configured buildings.	Building ID/filter.	Building metadata.	Role-dependent.	Unknown ID.
Floor	Retrieve floor config.	Building/floor ID.	Floor metadata/layout.	Role-dependent.	Parent relationship.
Room	Retrieve room metadata/current context.	Room ID.	Room metadata/status.	Role-dependent.	Configured room.

Category	Purpose	Input	Output	Auth.	Validation
Occupancy	Ingest/retrieve current/history.	Room, timestamp, state/count.	Accepted state/history.	Protected ingestion.	Count ≥ 0; valid timestamp.
Energy	Ingest/retrieve kW/kWh.	Room, timestamp, values.	Accepted state/history.	Protected ingestion.	Non-negative values.
Environment	Ingest/retrieve temperature/humidity/CO_2.	Room, timestamp, values.	Accepted state/history.	Protected ingestion.	Data-quality validation.
Resource State	Ingest/retrieve HVAC/lighting.	Room, timestamp, states.	Accepted state/history.	Protected ingestion.	Enum validation.
Alert	Retrieve/generate/ resolve.	Room/evidence/status.	Alert record/status.	Role-dependent.	Lifecycle validation.
Recommendation	Retrieve/create.	Issue/evidence/status.	Recommendation.	Role-dependent.	Supporting evidence.
Historical Analytics	24h/7d/30d or configured range.	Room/building/date range.	Aggregated time-series.	Role-dependent.	Date/range validation.
Simulation	Start/stop/configure.	Simulation config.	Simulation status.	Admin/Proposed.	Configuration validation.
IoT / Telemetry	Receive device telemetry.	MQTT message or payload.	Accepted/rejected status.	Service/device auth.	Schema/timestamp/ room/value validation.

10.1 API Response Strategy

Responses should use predictable JSON structures containing status, data and structured error information. Internal database errors, stack traces, credentials and implementation details should not be returned to the browser.

10.2 API Performance

Historical APIs should return aggregated data for charts where possible. Room/building queries should avoid unnecessary repeated database retrieval. Pagination may be used for large alert/history lists as an implementation technique.

11. DATA FLOW

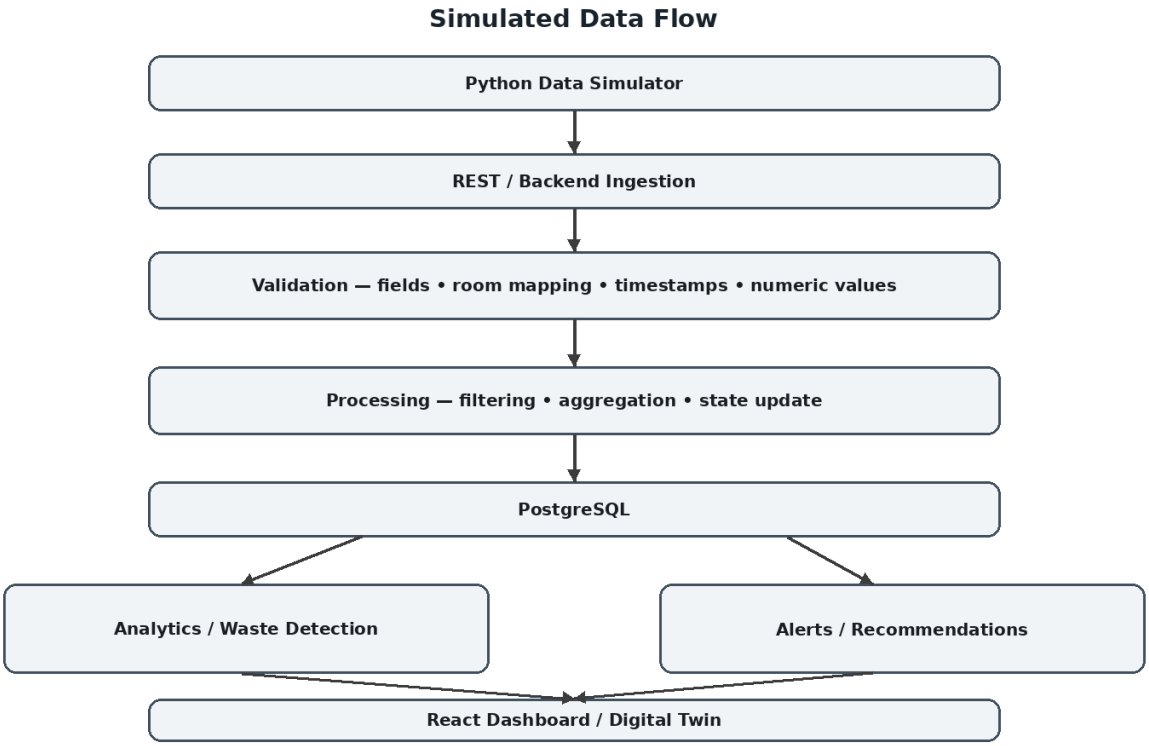


Figure 4. Simulated data flow.

11.1 Simulated Data Flow

1. The simulator generates schema-compatible occupancy, environmental, energy and resource-state records.
2. FastAPI ingestion receives the records and validates required fields and spatial identifiers.
3. Validated telemetry is stored in PostgreSQL and used to update current room state.
4. Processing services aggregate data and evaluate occupancy/resource/energy relationships.
5. Qualifying conditions create alerts and recommendations.
6. React retrieves current and historical state and renders the dashboard/Digital Twin.

11.2 Real IoT Data Flow

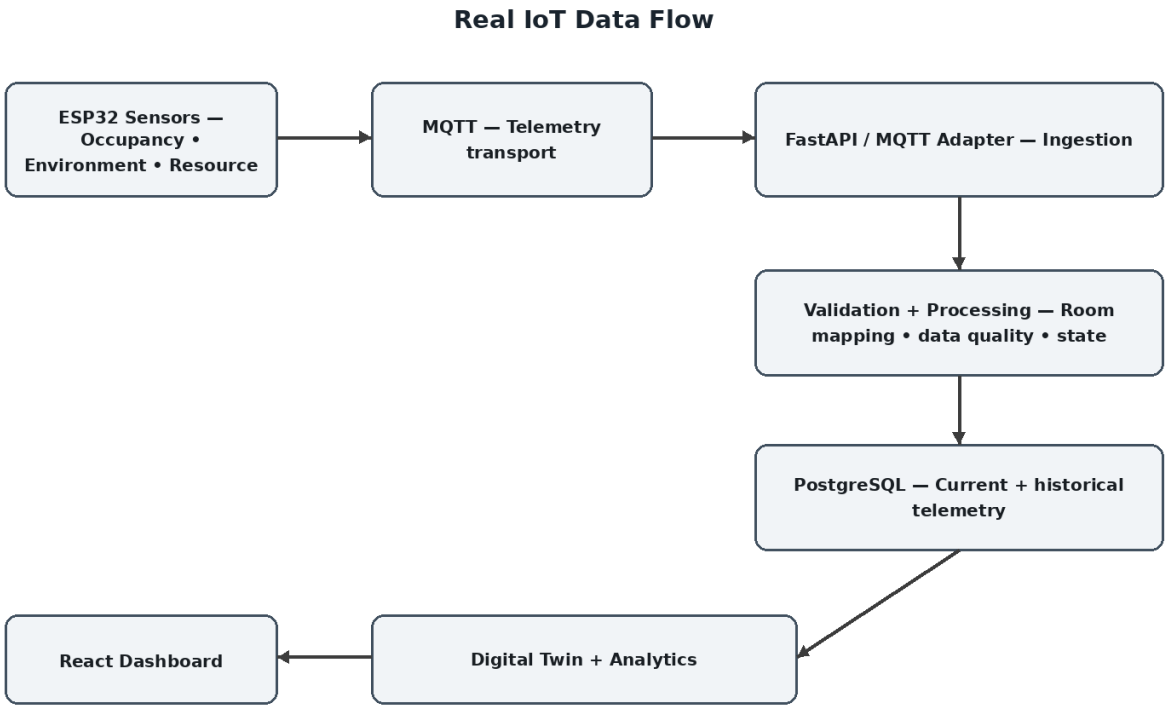


Figure 5. Real IoT data flow.

Where ESP32 hardware is available, sensors provide room telemetry to MQTT. A backend MQTT adapter maps messages to configured room/stream records, validates them and sends them through the same processing and storage path used by simulated data.

11.3 Hybrid Operation

Some rooms may use simulated data while selected rooms use real sensors. Both paths should produce the same logical telemetry fields. This follows the Phase-I hybrid approach and reduces dependence on full-room hardware deployment.

12. OCCUPANCY, ENERGY AND ENVIRONMENTAL DATA PROCESSING

Data	Processing	Aggregation / Presentation
Occupancy	Validate binary state/count; associate with room and timestamp; flag capacity anomalies.	Room → floor → building occupancy counts/percentages where required.
Temperature	Validate numeric reading and timestamp; retain latest valid value and history.	Room current value and historical trend.
Humidity	Validate numeric reading and timestamp.	Room current value and historical trend.
CO_2	Validate numeric reading and timestamp; compare with configured thresholds where applicable.	Room current value, historical trend and recommendation trigger.
Energy	Validate non-negative kW/kWh and timestamp.	Room/floor/building load and historical energy trend.
HVAC	Validate Active/Idle/OFF state and timestamp.	Room state and supported summaries.
Lighting	Validate ON/OFF/Dimmed state and timestamp.	Room state and waste-rule inputs.

12.1 Room State Composition

A room's displayed state combines spatial metadata with the latest valid telemetry from supported streams. Missing telemetry is not converted to zero. Each metric can independently display unavailable/stale status while other available metrics remain visible.

12.2 Historical Analytics

Historical analytics uses 24-hour, 7-day and 30-day occupancy-versus-energy analysis, with utilization/heatmap-style views where supported. Aggregation should occur in the backend to reduce browser payloads.

13. ENERGY-WASTE DETECTION ARCHITECTURE

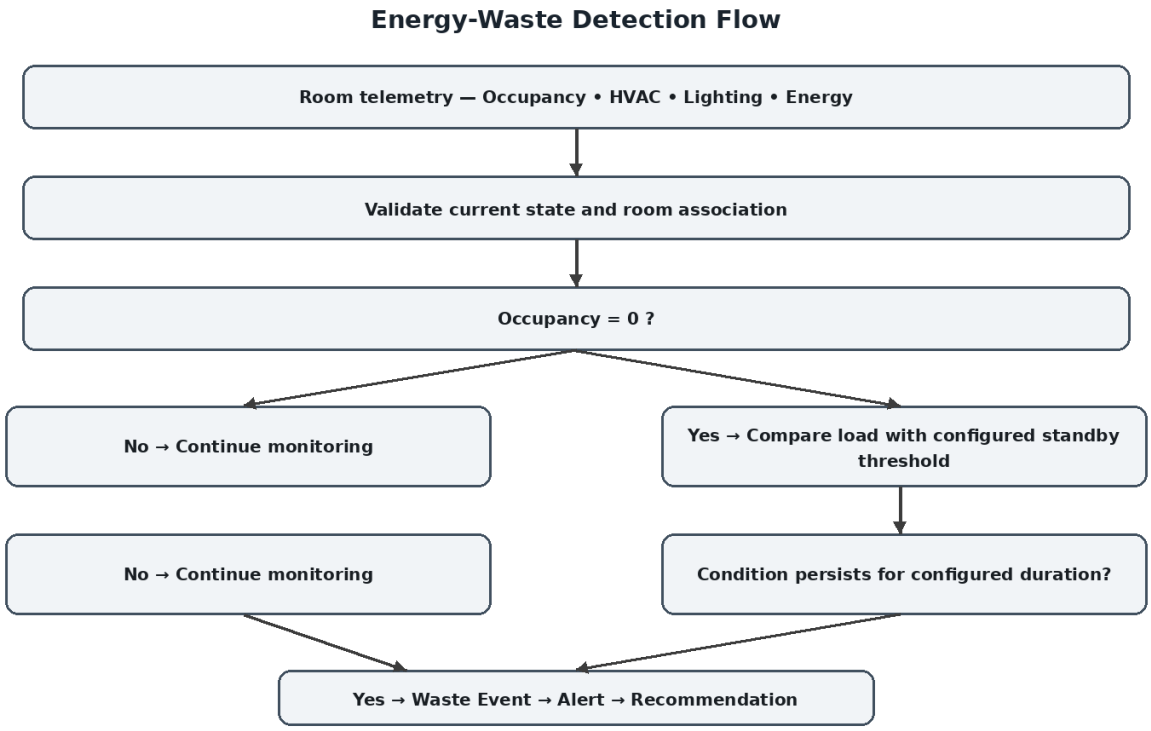


Figure 6. Energy-waste detection decision flow.

Step	Architecture Behavior
Receive telemetry	Obtain latest valid occupancy, energy, HVAC and lighting state for a room.
Validate	Ensure records belong to the configured room and have valid timestamps/values.
Check occupancy	If occupancy is non-zero, the empty-room waste rule does not qualify.
Compare load	For zero occupancy, compare active HVAC/lighting load against configured standby threshold(s).
Check duration	Require the qualifying condition to persist for the configured duration before alerting.
Create event	Persist waste alert with room, type, duration, severity, timestamp and estimated wasted kWh when calculable.
Recommend	Create human decision-support recommendation with supporting metrics.
Display	Expose alert/recommendation to authorized users; no actuator command is issued.

The PRD provides a 15-minute persistence example and an example threshold in one acceptance scenario. The SRS architecture treats thresholds and duration as configurable rather than inventing a universal value.

13.1 Decision-Support Boundary

The waste engine identifies situations for human attention. It shall not directly switch off or change HVAC, lighting, breakers, valves or other critical physical infrastructure.

14. AUTHENTICATION AND AUTHORIZATION ARCHITECTURE

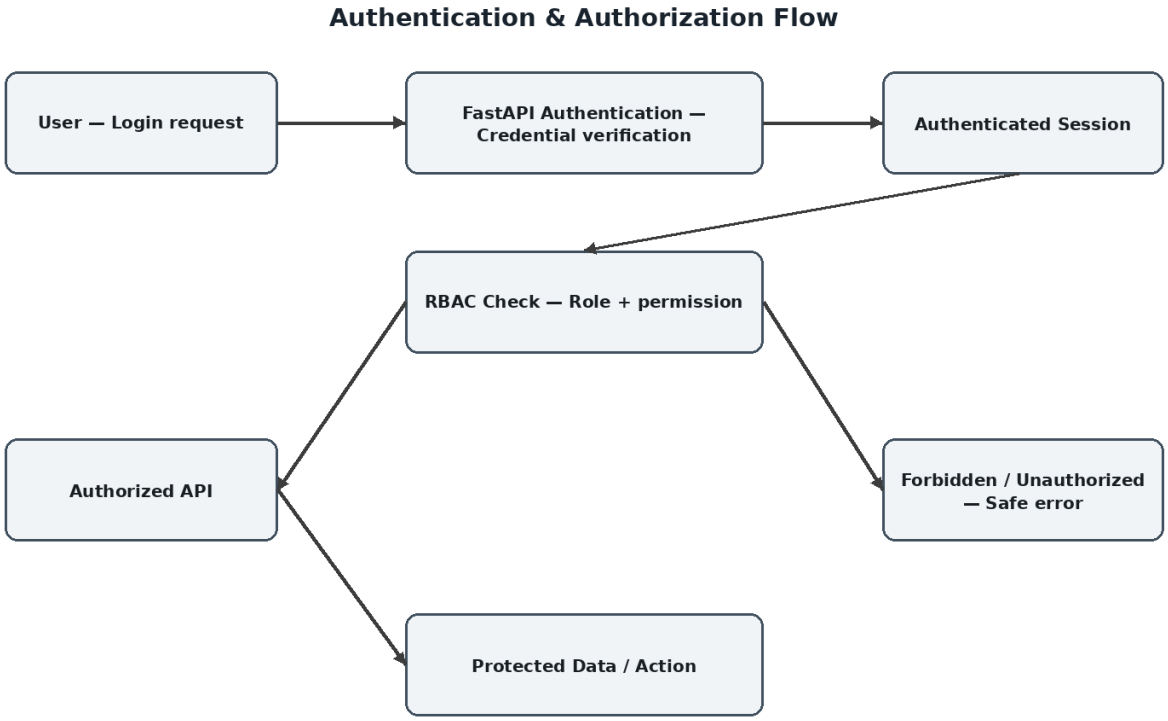


Figure 7. Authentication and authorization flow.

Role	Architecture Access Pattern
Facility / Building Administrator	Building-wide dashboard, spatial visibility, alerts and administrative capabilities defined by the SRS permission model.
Energy / Resource Manager	Energy/occupancy correlation, historical analysis, waste alerts and recommendations.
Campus Operations Staff	Operational waste alerts, room/resource status and inspection-oriented information.
Faculty / Students	Read-only room availability/current occupancy; other read-only details remain role-dependent.

14.1 Authorization Rules

- Backend authorization is authoritative; hiding a UI control is not sufficient protection.
- Protected API operations require authenticated context and appropriate role/permission.
- Unauthenticated requests are rejected or redirected to login.
- Authenticated users without permission receive a controlled forbidden response.
- Administrative configuration features are protected according to the SRS role matrix.
- Session expiry requires re-authentication.

15. SECURITY ARCHITECTURE

15.1 Confirmed Security Requirements

Control	Architecture Implementation
TLS 1.3	Use TLS 1.3 for applicable client/API and deployed service communication.
Authentication	Protected application/API operations require authenticated access.
RBAC	Roles are checked at backend/API boundaries.
Input validation	Telemetry, IDs, timestamps, numeric values and states are validated before persistence/processing.
API authorization	Protected ingestion and administrative operations require appropriate authorization.
Privacy	No PII, facial recognition, biometric tracking or individual identity tracking; occupancy is aggregate.

15.2 Recommended Security Practices

- Hash passwords with a modern adaptive password-hashing mechanism; never store plaintext passwords.
- Keep database, API, MQTT and sensor credentials outside source code.
- Use least-privilege database/service accounts.
- Use parameterized database operations to reduce injection risk.
- Return generic authentication/service errors; do not expose stack traces.
- Use secure HTTP headers and common web protections appropriate to the deployment.
- Log security events without storing credentials or unnecessary personal data.

16. DATA STORAGE AND RETENTION

Data	Storage Strategy	Retention
Building/Floor/Room metadata	PostgreSQL relational records.	Persistent for configuration.
User/account data	PostgreSQL account/role records or approved secure account store.	TBD by institutional/account policy.
Current telemetry/state	PostgreSQL latest-state/current-state representation.	Operational; exact policy TBD.
Historical telemetry	Timestamped PostgreSQL records for analytics and future analysis.	No fixed period in sources — TBD.
Alerts	PostgreSQL alert records.	TBD.
Recommendations	PostgreSQL decision-support records.	TBD.
Simulation data/logs	Application storage/logs as required for testing.	TBD.

No retention period is invented by this SAD. An approved retention policy should be defined before production-like deployment.

16.1 Query and Indexing Strategy

Telemetry queries should be indexed by room/stream and timestamp. Historical chart APIs should use time-bounded aggregation. Spatial lookup should use primary/foreign-key indexes. Alert retrieval should support room/status/timestamp filtering.

17. DEPLOYMENT ARCHITECTURE

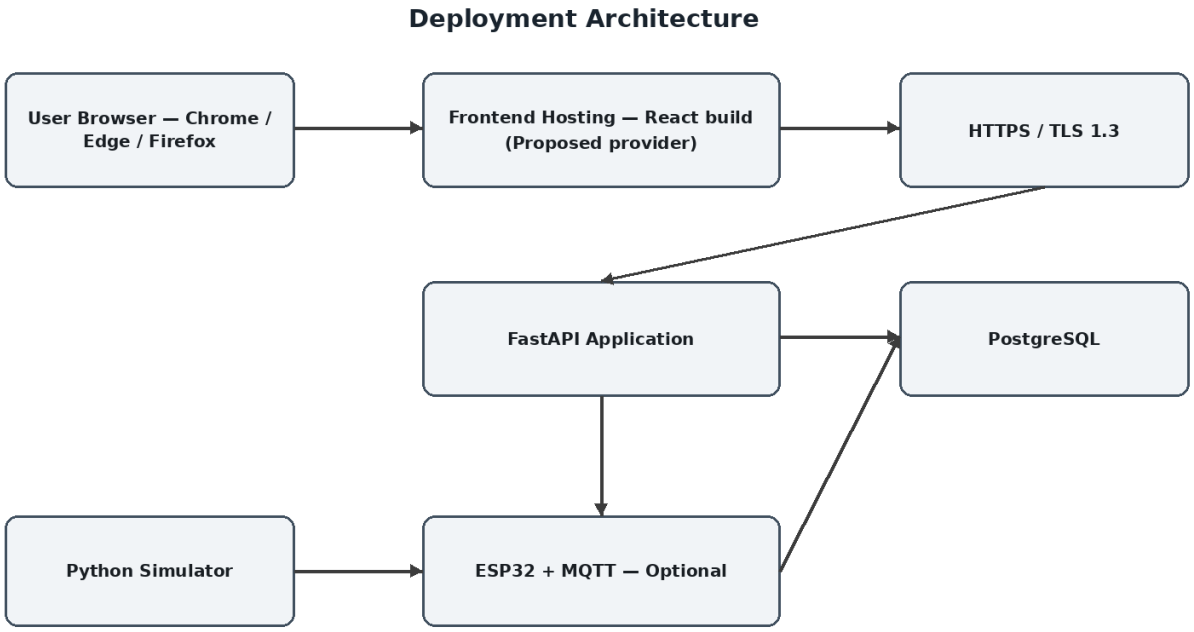


Figure 8. Practical deployment architecture.

Environment	Arrangement
Development	React development server + FastAPI development server + PostgreSQL + Python simulator on developer machine/approved development environment.
Demonstration	React frontend hosted or served as static build; FastAPI backend; PostgreSQL; simulator. ESP32/MQTT optional.
Production-like deployment	Browser → frontend hosting → FastAPI → PostgreSQL, with ESP32 → MQTT → backend if real telemetry is deployed.

17.1 Deployment Decisions

Element	Status
Frontend provider	Proposed — no provider fixed by source documents.
FastAPI provider	Proposed — no provider fixed.
PostgreSQL	Confirmed technology; local or managed deployment depending on demonstration environment.
MQTT broker	Proposed for real IoT deployment; exact broker not fixed.
DNS/domain	Proposed only if public demonstration requires it.

17.2 Local Demonstration

A faculty demonstration can run the complete application on one development machine with React, FastAPI, PostgreSQL and the simulator. This minimizes deployment risk while preserving the same logical architecture as the hosted arrangement.

18. MONITORING AND LOGGING

Area	Observe	Response
API health	Service availability/health endpoint.	Show status; investigate process/deployment failure.
Database	Connection success/failure and query errors.	Log dependency failure; return safe service error.
Telemetry ingestion	Accepted/rejected records, source status, latest timestamp.	Show stale/offline state and investigate source.
MQTT	Connection state/message receipt when enabled.	Reconnect/log failure; never fabricate telemetry.
API errors	4xx/5xx patterns and repeated failures.	Review validation/authorization/dependency logs.
Authentication	Failed login/forbidden requests.	Log security event without credentials.
Validation	Invalid values, room IDs, timestamps, duplicates.	Record reason; reject/quarantine.
Alert generation	Rule evaluation and persistence failures.	Log failure; do not silently discard qualifying evidence.
Performance	Ingestion-to-UI latency and chart render time.	Measure against SRS targets.

Lightweight application logs and health checks are sufficient for the current prototype. Centralized enterprise observability is not required unless later scale justifies it.

19. PERFORMANCE AND SCALABILITY

Requirement	Architecture Response
Dashboard update within 2 seconds	Use focused room-state responses, efficient queries/indexes and incremental frontend updates.
30-day chart within 1.5 seconds	Use aggregated historical queries and limit browser payloads.
500 rooms at 5-second sampling	Use indexed timestamped telemetry, efficient validation, batching where useful and separate ingestion/query paths within the modular backend.
99% uptime during 07:00–22:00	Use stable deployment, dependency health checks, controlled errors and database monitoring.
1920×1080 and 1366×768	Use responsive desktop layout and test major screens at both resolutions.

19.1 Scaling Path

The first scaling step is database/query optimization, not service decomposition. If later measurements reveal bottlenecks, selected ingestion or analytics modules can be separated. No such decomposition is required for the current MVP.

19.2 Building Scalability

Because building, floor and room identities are database/configuration records, additional buildings can be introduced without cloning frontend code or creating a separate backend per building.

20. ERROR HANDLING AND RESILIENCE

Condition	Architecture Handling
Invalid telemetry	Reject/quarantine at validation; log reason; do not update valid state.
Missing telemetry	Retain last valid state where appropriate and mark current metric stale/unavailable.

Condition	Architecture Handling
Duplicate telemetry	Detect using stream/timestamp/value identity or equivalent ingestion key.
Sensor offline	Expose offline/stale status; do not infer zero occupancy or energy.
MQTT failure	Log connection failure/retry state; continue serving available data.
API failure	Return structured error; frontend shows retryable state.
Database unavailable	Return controlled dependency error without internal database details.
Network timeout	Abort/retry according to client policy; show timeout state.
Invalid authentication	Reject with generic authentication failure.
Expired session	Invalidate protected context and require re-authentication.
Invalid building/floor/room	Return not-found/validation response; frontend shows error/no-data state.

20.1 Resilience Pattern

Detection → Logging → Safe Response → User Feedback. Valid data is preserved, missing values are not fabricated, and recommendations are generated only when supporting evidence is valid.

21. ARCHITECTURE DECISIONS AND TRADE-OFFS

Decision	Status	Approach	Reason / Trade-off
Frontend	Confirmed	React.js	Matches technology baseline and modular web UI needs.
Backend	Confirmed	Python + FastAPI	Matches Phase-I feasibility and API/processing needs.
Database	Confirmed	PostgreSQL	Structured relationships and time-stamped telemetry suit the model.
API	Confirmed / Proposed detail	REST-style application APIs	Simple integration; exact endpoint names remain Proposed.
IoT	Phase-I supported	MQTT for ESP32	Suitable telemetry transport; only when hardware is deployed.
Simulation first	Confirmed capability	Python simulator	Reduces hardware dependency and supports full software testing.
Backend style	Proposed	Modular monolith	Lower operational complexity than microservices.
Spatial representation	TBD	2D MVP vs 3D Digital Twin	PRD and Phase-I differ; same spatial model can support either renderer.
Spatial assets	PRD/SRS supported	SVG/JSON floor layouts	Simpler than heavy BIM/CAD pipeline for MVP.
Cloud provider	TBD/Proposed	Provider-independent	No vendor is fixed in sources.
Automatic control	Confirmed boundary	Not implemented	Monitoring/decision support only.
State management library	Not required	React feature/local state	Avoids unnecessary dependency.
Microservices/Kubernetes	Not selected	Single modular backend	Not needed for current scale.

21.1 Scope Clarification

The principal unresolved decision is the visual Digital Twin representation. The PRD defines a 2D MVP and explicitly excludes complex 3D/WebGL/VR, while the Phase-I report describes a 3D model and Three.js/React Three Fiber. The SRS records this as TBD. The architecture isolates the visualization renderer from the spatial/telemetry model so the renderer can be finalized without redesigning the core backend data architecture.

22. SCALABILITY AND FUTURE EXTENSIBILITY

Future Extension	Architecture Support
Additional buildings	Add Building records and associated floors/rooms; no new core service required.
Additional floors/rooms	Configuration/database additions using existing hierarchy.
More sensors	Add telemetry streams and ingestion mappings using the common schema.
Real ESP32 deployments	Enable MQTT adapter and map devices to configured room streams.
Occupancy prediction	Add Python/Scikit-learn analysis consuming historical occupancy data.
Advanced analytics	Add processing modules behind existing API boundary.
Predictive maintenance	Future approved module using equipment telemetry; not current MVP.
BMS integration	Future adapter for approved protocols such as BACnet/Modbus; control authority separately approved.
Campus-wide federation	Future multi-building configuration; not current PRD MVP.

The architecture should evolve by adding modules/adapters around the stable building-room-telemetry model rather than rewriting the core application.

23. FINAL ARCHITECTURE SUMMARY

CampusTwin EnergyMind uses a practical layered architecture centered on a React web application, a modular FastAPI backend, PostgreSQL persistence, and a common telemetry model that can receive simulated data today and selected ESP32/MQTT data later.

Flow	Architecture
Users	↓
React Dashboard / Digital Twin	Current state • building/floor/room navigation • analytics • alerts • recommendations
FastAPI APIs	Authentication • RBAC • application APIs • validation
Business Logic & Analytics	State processing • waste detection • historical analytics • recommendations
PostgreSQL	Spatial metadata • telemetry • resource state • alerts • recommendations • history
Simulator / REST / ESP32 + MQTT	Synthetic data first; real IoT integration where hardware/permissions permit

23.1 Final Architecture Characteristics

- Modular but not over-engineered.
- API-driven with clear presentation, application, processing and data responsibilities.
- Compatible with simulated and future real telemetry.
- Extensible from the three-building MVP to additional buildings/floors/rooms.
- Secure through authentication, RBAC, TLS 1.3 and input validation.
- Performance-oriented through indexed telemetry, aggregation and focused API responses.
- Explicitly limited to monitoring and decision support rather than autonomous physical control.
- Representation-neutral at the backend so the 2D/3D decision can be finalized without restructuring the core data architecture.

Architecture Baseline Status

Category	Status
React / FastAPI / PostgreSQL	Confirmed implementation baseline.
ESP32 / MQTT	Supported integration path; dependent on hardware/network/permissions.
Simulation	Required practical path for development/demo.
Three-building MVP	Current SRS baseline: SVH, Engineering Block, Law Block.
Additional buildings	Architecturally supported future extension.
2D vs 3D visualization	TBD; conflict explicitly retained from source documents.
Cloud provider	TBD / Proposed only.
Autonomous infrastructure control	Out of scope.

Appendix — Current MVP Room Mapping

Building	Floor	Room IDs
SVH	Ground	101–112
SVH	Second	201–213

Building	Floor	Room IDs
Engineering Block	Ground	101–108
Engineering Block	First	201–208
Engineering Block	Second	301–308
Engineering Block	Third	401–408
Law Block	Ground	101–108
Law Block	First	201–208
Law Block	Second	301–308
Law Block	Third	401–408

The current mapping contains 89 rooms across the three-building baseline. No additional room IDs are introduced by this architecture document.